

Development Environment Setup v1.0

1. 목적과 고정 원칙

본 시스템은 서버와 차량용 Android 클라이언트 간 통신만 제공한다. 브라우저용 관리자 웹과 외부 Tomcat/WAS는 구축 범위에서 제외한다.

- 백엔드: Spring Boot 실행 파일(JAR)의 내장 WAS를 컨테이너에서 실행외부 진입점: Nginx
- HTTPS 8443:443 단일 포트
- 클라이언트: Android 앱만 허용
- 내부 백엔드 포트: 8080(Compose 네트워크에서만 사용)데이터베이스: PostgreSQL 15
- 파일 수신/검증: Node.js watcher
- 컨테이너: Podman Compose

http://localhost:8080 또는 /sht_webapp/은 지원 URL이 아니다. PC 테스트는 https://localhost:8443, Android Emulator 테스트는 https://10.0.2.2:8443을 사용한다.

2. 디렉터리

```
D:\PROJECT\SmatBIS_jangsung\  
├server/  
│  ├──nginx/  
│  │  ├──certs/          # server.crt, server.key (비공개)  
│  │  └nginx.conf  
│  ├──egov-backend/      # Spring Boot 프로젝트, JAR 생성  
│  └node-watcher/  
├upload-store/           # 수신 전용 공간  
├client-android/  
├docker-compose.yml  
└.env                    # 비밀번호 등 로컬 비밀값, Git 제외
```

3. 실행 구조

Spring Boot, Node watcher, PostgreSQL, Nginx를 모두 컨테이너로 실행한다. 외부 공개 포트는 Nginx의 8443 하나이며, 백엔드 8080, watcher 3000, PostgreSQL 5432는 Compose 내부 네트워크에서만 사용한다.

```
services:  
  nginx:  
    image: nginx:alpine  
    ports:  
      - "8443:443"  
    volumes:  
      - ./server/nginx/nginx.conf:/etc/nginx/nginx.conf:ro  
      - ./server/nginx/certs:/etc/nginx/certs:ro  
    depends_on:
```

- egov-backend

```
egov-backend:
build:
  context: ./server/egov-backend
expose:
  - "8080"
  env_file: .env
  depends_on:
    - postgres-db

node-watcher:
build:
  context: ./server/node-watcher
env_file: .env
volumes:
  - ./upload-store:/app/upload-store
depends_on:
  - postgres-db

postgres-db:
  image: postgres:15-alpine
  env_file: .env
  volumes:
    - db-data:/var/lib/postgresql/data

volumes:
  db-data:
```

.env에는 POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB를 정의한다. network_mode: host는 Windows/Podman Desktop 환경의 동작 차이를 피하기 위해 사용하지 않는다.

4. Spring Boot 설정

application.properties:

```
server.port=8080
server.servlet.context-path=/api
spring.datasource.url=jdbc:postgresql://postgres-db:5432/smatbis_db
```

백엔드 API는 /api/...로만 제공한다. 관리용 HTML 화면, /sht_webapp, 외부 8080 접근은 구현하지 않

는다.개발 실행:

```
./mvnw clean package
podman compose build egov-backend
podman compose up -d postgres-db egov-backend
```

Spring Boot 컨테이너용 Dockerfile 생성:

```
cd /d/PROJECT/SmatBIS_jangsung
cat > server/egov-backend/Dockerfile <<'EOF'
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
EOF
```

5. Nginx 설정 계약

```
events { worker_connections 1024; }
http {
    include mime.types;
    server {
        listen 443 ssl;
        server_name localhost;
        ssl_certificate /etc/nginx/certs/server.crt;
        ssl_certificate_key /etc/nginx/certs/server.key;

        location /api/ {
            proxy_pass http://egov-backend:8080/api/;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-Proto https;
        }

        location / {
            return 404;
        }
    }
}
```

Spring Boot는 개발과 운영 모두 컨테이너로 실행하며 Nginx는 egov-backend:8080으로 연결한다.

6. 사설 CA

서버 인증서 SAN에는 localhost, 127.0.0.1, 10.0.2.2를 포함한다. 생성한 rootCA.crt는 Android 앱의 res/raw에 포함하고, 개인키(rootCA.key, server.key)는 Git에 저장하지 않는다. Android의 인증서 검증은 CA 신뢰 체인과 공개키 고정 정책을 함께 검토한다.

7. Android 설정

- 서버 기본 URL: https://10.0.2.2:8443
- API 예: GET /api/v1/content/sync?
- vehicleId=...인증서: rootCA.crt 내장

- FHD 에뮬레이터: 1920x1080
- Room WAL, 임시 파일 다운로드 후 원자적 rename 적용

8. 구축 및 테스트 순서

1. PostgreSQL, Spring Boot, Node watcher, Nginx 이미지 생성
2. Compose 네트워크에서 전체 서버 기동
3. podman compose ps로 컨테이너 상태 확인
4. GET https://localhost:8443/api/actuator/health로 외부 TLS 경로 확인
5. Android Emulator에서 https://10.0.2.2:8443/api/... 호출
6. 인증서 오류, 404, 502, DB 연결 실패를 각각 분리해 기록

성공 기준은 브라우저 화면 표시가 아니라 Android API 호출의 TLS·인증·JSON 응답 성공이다.

9. Git Bash 구축 명령

```
cd /d/PROJECT/SmatBIS_jangsung
mkdir -p server/nginx/certs upload-store
podman compose build
podman compose up -d
podman compose ps
curl -k https://localhost:8443/api/actuator/health
```

로그 확인 및 종료:

```
podman compose logs -f egov-backend
podman compose logs -f nginx
podman compose down
```